7,5/10 Good job!

stk4290

Mandetory assignement stk4290

1.a, 2/2

$$I(X;Y,Z) = \sum x,y,zPX,Y,Z(x,y,z), \log \frac{P_{X,Y,Z}(x,y,z)}{P_X(x) * P_Y(y,z)}$$

$$I(X;Y,Z) = \sum_{x,y,z} P_{X,Y,Z}(x,y,z) \log \frac{P_{X,Y,Z}(x,z|y) * P(y)}{P_X(x) * P_Y(z|y)P(y)}$$

$$I(X;Y,Z) = \sum_{x,y,z} P_{X,Y,Z}(x,y,z) \log \frac{P_{X,Y,Z}(x,z|y) * P(y)}{P_X(x) * P(y) * P(z|y)}$$

$$I(X;Y,Z) = \sum_{x,y,z} P_{X,Y,Z}(x,y,z) \log \frac{P_{X,Y,Z}(x,z|y) * \frac{P(y|x)*P(x)}{P(x|y)}}{P_X(x) * P(y) * P(z|y)}$$

$$I(X;Y,Z) = \sum_{x,y,z} P_{X,Y,Z}(x,y,z) \log \frac{P_{X,Y,Z}(x,z|y) * P(y|x) * P(x)}{P_X(x) * P(y) * P(z|y)P(x|y)}$$

$$I(X;Y,Z) = \sum_{x,y,z} P_{X,Y,Z}(x,y,z) \log \frac{P_{X,Y,Z}(x|z,y) * P(y|x) * P(x)}{P_X(x) * P(y) * P(x|y)}$$

$$I(X;Y,Z) = \sum_{x,y,z} P_{X,Y,Z}(x,y,z) \log \frac{P(y|x) * P(x)}{P_X(x) * P(y)} + \sum_{x,y,z} P_{X,Y,Z}(x,y,z) \log \frac{P_{X,Y,Z}(x|z,y)}{P(x|y)}$$

$$I(X;Y,Z) = \sum_{x,y} P_{X,Y}(x,y) \log \frac{P(x,y)}{P_X(x) * P(y)} + \sum_{x,y,z} P_{X,Y,Z}(x,y,z) \log \frac{P_{X,Y,Z}(x|z,y)}{P(x|y)}$$

1p

$$I(X;Y,Z) = I(X;Y) + I(X;Z|Y)$$

1.b

$$I_P(X;Y) \leq I(X;Y;Z)$$

$$0 \leq I(X;Y,Z) - I_P(X;Y)$$

using the fact that $log(z) \geq 1 - \frac{1}{z}$

$$I(X;Y,Z) - I_P(X;Y) = \sum_{x,y} P_{X,Y}(x,y,z) \log \frac{P(x,y,z)}{P_X(x) * P(y,z)} - \sum_{x,y} P_{X,Y}(x,y) \log \frac{P(x,y)}{P_X(x) * P(y)}$$

$$I(X;Y,Z) - I_P(X;Y) \geq \sum_{x,y,z} P_{X,Y,Z}(x,y,z)\,(1 - \frac{P_X(x) * P(y,z)}{P(x,y,z)}) - \sum_{x,y,} P_{X,Y}(x,y)\,(1 - \frac{P_X(x) * P(y)}{P(x,y)})$$

$$\sum_{x,y,z} P_{X,Y,Z}(x,y,z) \; - \sum_{x,y,z} P_X(x) * P(y,z) - \sum_{x,y,} P_{X,Y}(x,y) \; + \sum_{x,y,} P_X(x) * P(y)$$

$$1 \; - \sum_{x,y,z} P_X(x) * P(y,z) - 1 \; + \sum_{x,y} P_X(x) * P(y)$$

$$1 \; + \sum_{x} P_X(x)(\sum_{y} P(y) - \sum_{y,z} P(y,z)) - 1 \; = 0$$

which mean:

$$I(X;Y,Z) - I_P(X;Y) \geq 0$$

or

1p

$$I(X;Y,Z) \geq I_P(X;Y)$$

1.5

2.a,

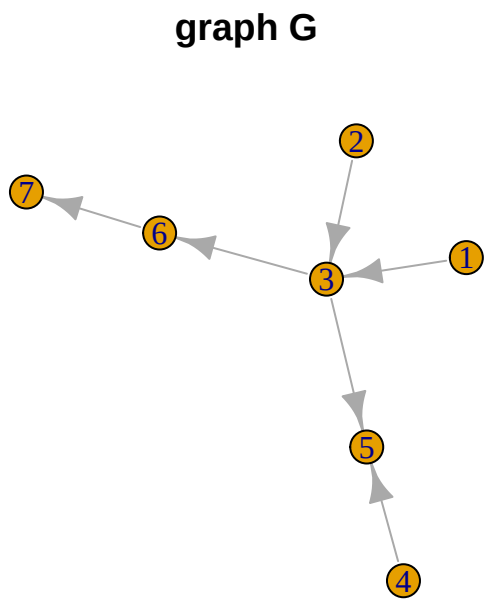
$$P(A,B,D,E,F,G) = \sum_{C} P(A,B,D,E,F,G|C) * P(C)$$

graph G factorizing the joint in this manner:

$$P(A,B,C,D,E,F,G) = P(A) * P(B) * P(C|A,B) * P(D) * P(E|D,C) * P(F|C) * P(G|F)$$

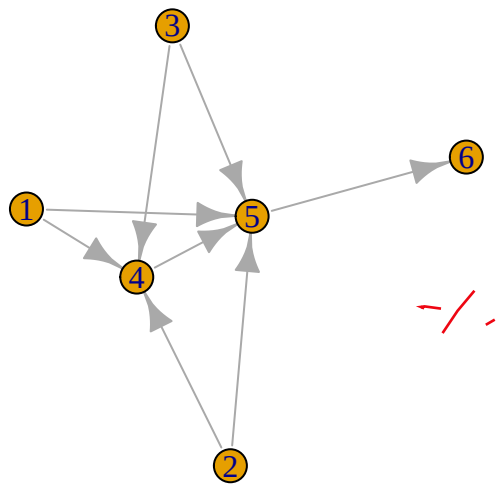
In the plottet Dags: 1,2,3,4,5,6,7 corresponds to A,B,C,D,E,F,G.

```
library(igraph)


# Define graphs
G1 <- matrix(0,7,7)
G1[1,3] <- 1
G1[2,3] <- 1
G1[3,5] <- 1
G1[4,5] <- 1
G1[3,6] <- 1
G1[6,7] <- 1


plot(graph_from_adjacency_matrix(G1))
title("graph G")
```




```
G1 <- matrix(0,6,6)
G1[1,5] <- 1
G1[1,4] <- 1
G1[1,5] <- 1
G1[2,5] <- 1
G1[2,4] <- 1
G1[3,4] <- 1
G1[5,6] <- 1
G1[4,5] <- 1
G1[3,5] <- 1


plot(graph_from_adjacency_matrix(G1),col="blue")
title("minimal I map for graph G_marked")
```

minimal I map for graph G_marked



-/.

1p

Removing C.Activates the path between:

(A,F),(A,E) (B,F),(B,E) (E,F)

Using the the minimal I-map algorithm,where one has to check. First you choose an ordering.In this case the ordering is A,B,D,E,F,G.

Then you add each node into a list,according to the ordering.For each node try to find a minimal parent set U from the previous nodes in the ordering which makes

the node independent from all other nodes that are not included in U,given U. You check these conditional independt querys by looking at the original DAG.

$$X_i \perp X_{-i} - U|U$$

The fact that,now E,A,B is parent of F.We see that D cannot be independent of F.In this Case $X_{-i} - U$ must be empty.That means That D also is a parent of F.

The last plotted dag is the minimal I-map of G'.Removing any of the edges in this DAG,makes it not an I-map of the distribution generated by G'.

2.b

We want to show that:

$I_l(\mathcal{H}) = \{(X \perp \mathcal{X} - \{X\} - MB_{\mathcal{H}}(X_i)| \quad MB_{\mathcal{H}}(X_i)) \quad : X \in \mathcal{X}\}$

implies

$$I_p(\mathcal{H}) = \{(X \perp Y | \mathcal{X} - \{X, Y\}) \quad : X - Y \notin \mathcal{H}\}$$

we choose a

$Y : Y \notin MB_{\mathcal{H}}(X)$

we have

$\mathcal{X} = \{\mathcal{X} - Y\} \cup \quad Y$

$$I_l(\mathcal{H}) = \{(X \perp Y, \mathcal{X} - \{X, Y\} - MB_{\mathcal{H}}(X_i) | \quad MB_{\mathcal{H}}(X_i)) \quad : X \in \mathcal{X}\} \quad \Rightarrow$$

$$\cancel{I_l(\mathcal{H})} = \{(X \perp Y | \mathcal{X} - \{X, Y\} - MB_{\mathcal{H}}(X_i) \quad + MB_{\mathcal{H}}(X_i)) \quad : X \in \mathcal{X}\}$$

Using weak union we have:

at which step is the weak union axiom used?

$I_l(\mathcal{H}) \Rightarrow I_p(\mathcal{H}) = \{(X \perp Y | \mathcal{X} - \{X, Y\}) \quad : X - Y \notin \mathcal{H}\}$

3.a. 1/2

Using the hint we have that.

$$\frac{P(X_2 = 0 | X_1 = 1, X_3 = 0)}{P(X_2 = 1 | X_1 = 1, X_3 = 0)} = \frac{P(X_2 = 0 | X_1 = 1, X_3 = 1)}{P(X_2 = 1 | X_1 = 1, X_3 = 1)}$$

and

$$\frac{P(X_2 = 0 | X_1 = 0, X_3 = 0)}{P(X_2 = 1 | X_1 = 0, X_3 = 0)} = \frac{P(X_2 = 0 | X_1 = 0, X_3 = 1)}{P(X_2 = 1 | X_1 = 0, X_3 = 1)}$$

solving for the parameters for the first condition:

$$\frac{P(X_2 = 0 | X_1 = 1, X_3 = 0)}{P(X_2 = 1 | X_1 = 1, X_3 = 0)} = \frac{P(X_2 = 0 | X_1 = 1, X_3 = 1)}{P(X_2 = 1 | X_1 = 1, X_3 = 1)}$$

using Bayes formula:

Def. of cond. probability

$$\frac{\frac{P(X_2=0, X_1=1, X_3=0)}{P(X_1=1, X_3=0)}}{\frac{P(X_2=1, X_1=1, X_3=0)}{P(X_1=1, X_3=0)}} = \frac{\frac{P(X_2=0, X_1=1, X_3=1)}{P(X_1=1, X_3=1)}}{\frac{P(X_2=1, X_1=1, X_3=1)}{P(X_1=1, X_3=1)}}$$

$$\frac{\frac{e^{\phi_1}}{Z}}{\frac{e^{\phi_1+\phi_2+\phi_{12}}}{Z}} = \frac{\frac{e^{\phi_1+\phi_3+\phi_{13}}}{Z}}{\frac{e^{\phi_1+\phi_2+\phi_{12}+\phi_3+\phi_{13}+\phi_{123}}}{Z}}$$

$$\frac{\exp(\phi_1)}{\exp(\phi_1 + \phi_2 + \phi_{12})} = \frac{\phi_1 + \phi_3 + \phi_{13}}{\phi_1 + \phi_2 + \phi_{12} + \phi_3 + \phi_{13} + \phi_{123}}$$

doing the same for:

$$\frac{P(X_2 = 0 | X_1 = 0, X_3 = 0)}{P(X_2 = 1 | X_1 = 0, X_3 = 0)} = \frac{P(X_2 = 0 | X_1 = 0, X_3 = 1)}{P(X_2 = 1 | X_1 = 0, X_3 = 1)}$$

we get:

$$\frac{1}{\phi_2} = \frac{\phi_3}{\phi_2 + \phi_3 + \phi_{23}}$$

$$\phi_2 = \frac{\phi_2 + \phi_3 + \phi_{23}}{\phi_3}$$

the restrictions imply that the parameters must satisfy both.

$$\phi_2 = \frac{\phi_2 + \phi_3 + \phi_{23}}{\phi_3}$$

and

$$\frac{\phi_1}{\phi_1 + \phi_2 + \phi_{12}} = \frac{\phi_1 + \phi_3 + \phi_{13}}{\phi_1 + \phi_2 + \phi_{12} + \phi_3 + \phi_{13} + \phi_{123}}$$

3.b

Here we only get one restriction which forces the parameters to satisfy:

$$\frac{\phi_1}{\phi_1 + \phi_2 + \phi_{12}} = \frac{\phi_1 + \phi_3 + \phi_{13}}{\phi_1 + \phi_2 + \phi_{12} + \phi_3 + \phi_{13} + \phi_{123}}$$

4.a 3/4

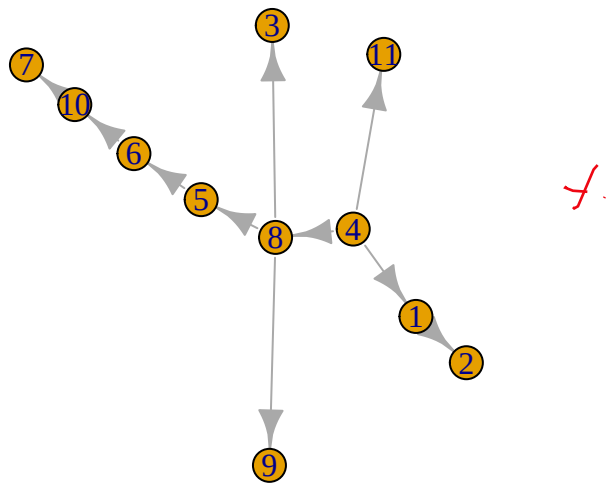
```
library(dplyr)
library(readr)

data=read.csv("bn_data.csv",header = TRUE)


#Creating combinations of parents and child
j_con1=combn(1:11,2)
j_con2=combn(11:1,2)

j_con=t(cbind(j_con1,j_con2))

#Empty matrix for calculating the marginal empirical distribution
emp_dist=matrix(rep(rep(0,11),3),ncol = 3,nrow = 11)

#function for calculating marginal empirical dist
empircal <- function(emp_dist,data){


  for(i in 1:3){


    emp_dist[,i]=colSums(data==i-1)/length(data[,1])
  }

  return(emp_dist)
}
```

```r
#Dist for current data
empir=empircal(emp_dist,data)

# function for calculating joint empirical dist
Joint_P=function(config,data,i){


  # for each combination count how many time each joint configuration
  #appears in the two columns of interest in the data
  count=data[,c(j_con[i,])] %>% group_by_all %>% count

  count[3]=count[3]/sum(count[3])


  return(count)


}

joint=Joint_P(config,data,3)

#Function for calculating the familiy-score
Fam_score= function(empir,joint){
  #nr of joint config for each parnet-child configuration
  r=as.matrix(joint[,3])
  #empty vec
  like_vec=rep(0,length(r))
  inf=0
  #print(length(r))
  for(i in 1:length(r)){
    #extracting which nodes in parent child configuration
    which_nodes_j=readr::parse_number(names(joint)[1:2])

    #First element in i-th joint configuration
    which_val_j_1=as.numeric(joint[1:2][i,1])

    #Second element
    which_val_j_2=as.numeric(joint[1:2][i,2])

    #Both elements together
    which_val=which(joint[1]==which_val_j_1 & joint[2]== which_val_j_2)

    #Which probability does the joint config correspond to
    spe_joint=joint[3][[1]][which_val]

    # Calculating mutual information for each joint config
    like_vec[i]=spe_joint*log(spe_joint/(empir[which_nodes_j[1],which_val_j_1+1]*empir[which_nodes_j[2]
    which_val_j_1=0
    which_val_j_2=0
  }
  inf=sum(like_vec)

  #Summing them up
```

```
  like=length(data[,1])*(inf)

  return(cbind(like,which_nodes_j))
}

#Calculating famscore for all parent child config
all_joints=matrix(rep(0,440),nrow = 110,ncol = 4)
for(i in 1:110){
  j=Joint_P(config,data,i)
  Fam=Fam_score(empir,j)

  all_joints[i,]=c(Fam)



}

#Two columns are equal,delting the sencond.Happens because
#the use of cbind as a return value.Cbinding a number and a two
#Dimensional vector
all_joints=all_joints[,-2]









library(prodlim)

#Maximum tree algorithm
maximum_tree=function(all_joints){
  #putting first element as 4
  node=4
  #into the tree list
  nodes=c(node)

  edge=c()
  final=c()



  #While tree list length is less then number of nodes
  while(length(nodes)<11){

    #The allowed search space of where to put an edge is
    #From nodes that are in the tree list to nodes that are not
    #For each iteration creating this search space
    edge=expand.grid(nodes,setdiff(1:11,nodes))

    #converting it into  a data frame
    edge=data.frame(edge)
```

```
    colnames(edge)=c("X1","X2")

    #The goal know is to extract the rows in the fam-weight-matrix
    #That corresponds to the legal search-space
    resticted=data.frame(all_joints[,2:3])
    colnames(resticted)=c("X1","X2")
    #Which is done here
    element_max_list=row.match(edge,resticted)

    #Now Extracting these rows
    max_elemnet_list1=all_joints[element_max_list,]

    #Checking which of them has the biggest fam-score
    max_element_sub=which.max(max_elemnet_list1[,1])

    #Choosing that edge
    max_comb=max_elemnet_list1[max_element_sub,2:3]

    #Second element in edge is the element added to tree-list
    best_neig=max_comb[2]
    nodes=c(nodes,best_neig)

    #add edge to edge list
    final=c(final,c(max_comb))

  }
  return(final)
}

#Calling function
s=maximum_tree(all_joints)

#Creating empty adjecent matrix for tree
tree=matrix(rep(0,121),nrow = 11,ncol=11)

#Odd equal parent
parent=s[seq(1,19,2)]

#Even equal child
child=s[seq(2,20,2)]


index=cbind(parent,child)
input=0

#Filling up the adjecent matrix

 for(i in 1:10){
   input=as.matrix(index[i,])

   tree[input[1],input[2]]=1

 }
```

```r
library(igraph)

#Plotting graph
plot(graph_from_adjacency_matrix(tree))
```



4.b

Calculation for mutual information:

```r
#Using the edges from the tree
index=cbind(parent,child)
relevant_edge=index
relevant_edge=data.frame(relevant_edge)
colnames(relevant_edge)=c("X1","X2")
resticted_1=data.frame(all_joints[,2:3])
colnames(resticted_1)=c("X1","X2")

#Extracting relevant index
Mutual_info_index=row.match(relevant_edge,resticted_1)

Mutual_info=sum(all_joints[Mutual_info_index,1])

Mutual_info
```

```
## [1] 1.749441
```

Calculation for entropy and the likelihood:

```
entrop=function(empir,node){

entropy=sum(-empir[node,]*log(1/empir[node,]))


   return(entropy)
}

#
entropy=0
for(i in 1:11){
entropy=entropy+entrop(empir,i)}

entropy
```

```
## [1] -9.408859
```

```
M=1000
likelihood=M*(Mutual_info+entropy)
likelihood
```

```
## [1] -7659.418
```

To calculate the BIC one need to subtract

$$\frac{\log(M)}{2} * Dim(Tree)$$

M=1000 Dim(Tree)=11

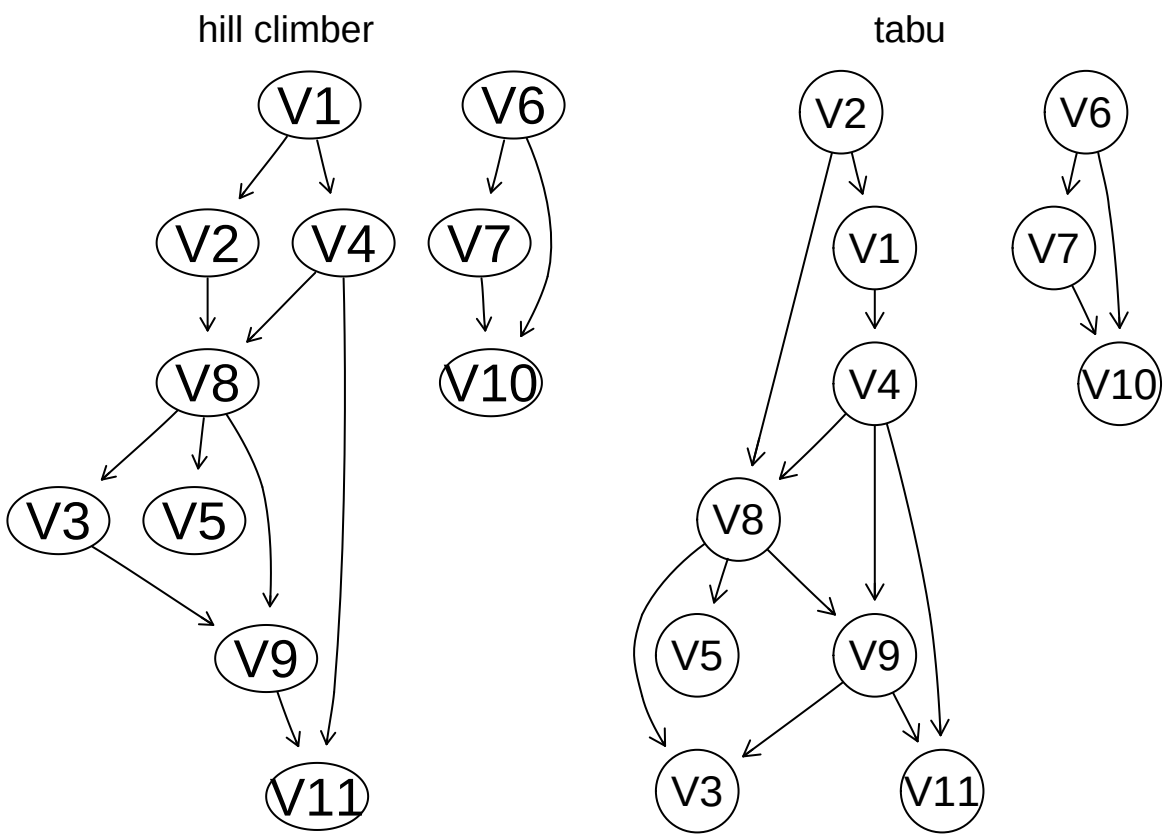
```
Bic_constant=(log(M)/2)*11
Bic=likelihood-Bic_constant

Bic
```

```
## [1] -7697.41
```

4.c

```
library(bnlearn)
library(BiocManager)

df=data.frame(data)

cols_df <- c("V1","V2","V3","V4","V5","V6","V7","V8","V9","V10","V11")
df[,cols_df] <- lapply(df[,cols_df], as.factor)
par(mfrow = c(1, 2))
#  hill climber algorithm
```

```r
hc_df = hc(df)
graphviz.plot(hc_df, shape="ellipse", layout = "dot", main = "hill climber")
hc_score = score(hc_df, df, type = "bic")

# tabu algorithm
tabu_df = tabu(df)
graphviz.plot(tabu_df, layout = "dot", main = "tabu")
```



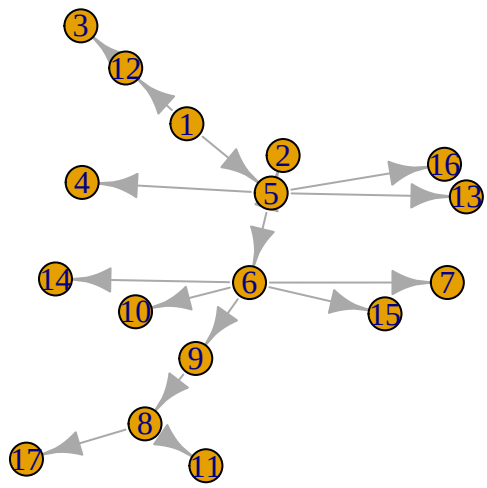

```r
tabu_score = score(tabu_df, df, type = "bic")



cat("BIC score of hill climber ", hc_score, "\n", "BIC score of TABU ", tabu_score, "\n")
```

```
## BIC score of hill climber  -7730.793
##  BIC score of TABU  -7725.527
```

1p

4.d

Ran the same code



From The tree it looks like.The two nodes 12 and 5 blocks the path between all other nodes and 1.

Node 1 is binary voting:(Democrat,Repoblican)

Node 12 is binary "synfuels-corporation-cutback":(0,1) or (no,yes) node 5 is binary "physician-fee-freeze":(0,1) or (no,yes)

node 12 and 5 depends on the value of node 1.